

אוניברסיטת אריאל
הפקולטה למדעי ההנדסה
המחלקה להנדסת חשמל ואלקטרוניקה

מערכת זיהוי אותות FSK רב-ערוצית על FPGA
Multi-Channel FSK Signal Detection System on FPGA
(PYNQ-Z2, Zynq-7020)

ספר פרויקט גמר בהנדסת חשמל ואלקטרוניקה

מוגש על ידי: איתמר

בהדרכת: _____

יוני 2026

תקציר

מערכות תקשורת אלחוטיות מודרניות, וכן יישומים בתחומי האינטרנט של הדברים (IoT), חישה מבוצרת ומעקב ספקטרי, נדרשות לעבד אותות נכנסים בזמן אמת ולקבל החלטות במהירות גבוהה ובצריכת הספק נמוכה. אפנון תדר בינארי (Binary Frequency Shift Keying, BFSK) הוא אחת משיטות האפנון הפשוטות והעמידות ביותר בפני רעש, ולכן נפוץ בתקשורת נתונים בתדר נמוך, בחיישנים אלחוטיים ובמערכות זיהוי איתות. מערכים מבוססי FPGA, ובפרט פלטפורמות מסוג System-on-Chip המשלבות לוגיקה מתוכנתת (PL) עם מעבד מוטבע (PS) כגון משפחת Xilinx Zynq, מאפשרים מימוש של צנרות עיבוד אותות מקבילות וגמישות, יחד עם שכבת בקרה ותצוגה מבוססת תוכנה.

מערכות זיהוי FSK קיימות רבות מתמקדות בערוץ קליטה בודד, ומספקות מידע על איתור אנרגיה, פענוח סיביות וחילוץ כתובת עבור מקור אחד בלבד. בתרחישים מעשיים – כגון רשת חיישנים מבוצרת, מערכת ניטור ספקטרום רב-אפיקית, או מקלט עם כמה אנטנות/כניסות – יש צורך ביכולת לעבד מספר ערוצי קליטה במקביל, להשוות בין הכתובות שחולצו בכל ערוץ, ולבחור את הערוץ עם רמת האות האמינה ביותר. קיים אפוא פער בין הפתרונות החד-ערוציים הנפוצים לבין הצורך במערכת רב-ערוצית מקבילה, הניתנת להרחבה, הרצה על חמרה זולה ונגישה כגון PYNQ-Z2, ועם נראות מלאה לתהליך הזיהוי בזמן אמת.

בפרויקט זה תוכננה, מומשה ונבדקה מערכת FPGA רב-ערוצית לזיהוי אותות FSK, הכוללת לכל ערוץ צנרת עיבוד אותות מלאה: חיץ דגימות, גלאי אנרגיה, חישוב FFT בגודל 256 (באמצעות IP של Xilinx), בחירת שני אגפי תדר (bins) המתאימים לתדרי ה-FSK, חישוב הספק, החלטת סיבית (FSK Decision), גילוי מילת סנכרון (Sync Detector) באמצעות רגיסטר הסטה, וחילוץ שדה כתובת (Adders_Extractor). מעל הצנרות הללו נבנתה שכבה רב-ערוצית (multi_channel_top) הכוללת מודול matcher להשוואת כתובות בין ערוצים (match_detected), מודול selector לבחירת הערוץ עם מרווח ההספק הגבוה ביותר (best_channel), ומיתוג פלט (output mux) המאחד את כל הערוצים למילת AXI-Stream רחבה (64 סיביות) הנשלחת ל-PS באמצעות AXI DMA. לצורך בדיקה עצמית של המערכת ללא צורך במקור אות חיצוני, פותח מודול מחולל חבילות (fsk_packet_generator) המייצר אותות FSK סינטיים, כולל רעש בין חבילות וכתובות תואמות בחבילה הראשונה כדי להפעיל את מנגנון ה-match_detected. בצד התוכנה נכתבה אפליקציית Python (run_fpga.py) הרצה על ה-PS, הקוראת נתונים באמצעות AXI DMA, מפענחת את מבנה המילה ומציגה תוצאות ניטור גרפית בזמן אמת בסביבת Jupyter.

התכן אומת בסימולציה ובחמרה: ה-bitstream נבנה ב-Vivado 2019.1 עבור Zynq-7020 ועמד בדרישות התזמון (Worst Negative Slack חיובי), נצרב ל-PYNQ-Z2 ונבדק בפועל. נמדדה תקשורת תקינה בין שני ערוצי קליטה, חבילת הכתובת הראשונה הניבה התאמה בין הערוצים (match_detected=1), וערוצי ההספק, הסנכרון והכתובות הוצגו בזמן אמת בתצוגת הניטור הפיתונית. התוצאות מדגימות מימוש מעשי ומשתרע של מקלט FSK רב-ערוצי על חמרה לוגית מתוכנתת בעלת עלות נמוכה, המהווה בסיס להרחבות עתידיות כגון תמיכה במספר ערוצים גדול יותר, חילוץ פרוטוקול מורכב יותר ומדדי ביצועים סטטיסטיים (Pd, Pfa).

מילות מפתח:

FPGA, Zynq-7020, PYNQ-Z2, אפנון FFT, FSK, עיבוד אותות בזמן אמת, AXI DMA, זיהוי רב-ערוצי, גילוי סנכרון, Software-Defined Radio.

תוכן עניינים

תקציר	2
תוכן עניינים	3
רשימת תרשימים	5
רשימת טבלאות	5
פרק 1 – מבוא	6
1.1 רקע כללי ותחום הפרויקט	6
1.2 חשיבות הנושא ומגמות עכשוויות	6
1.3 מצב קיים וטכנולוגיות דומות	6
1.4 הפער הקיים והבעיה הנדונה	6
1.5 מטרת הפרויקט ומבנה הספר	7
פרק 2 – סקירת ספרות ורקע תיאורטי	8
2.1 אפנון FSK ועקרונות גילוי אנרגיה	8
2.2 עיבוד אותות מבוסס FFT על FPGA	8
2.3 סנכרון מסגרות (Sync Detection) וחילוץ כתובת	8
2.4 קבלה רב-ערוצית, בחירת ערוץ וגיוון (Diversity)	8
2.5 פלטפורמות Zynq/PYNQ ותכן משולב חמרה-תוכנה (HW/SW Co-Design)	9
2.6 סיכום הסקירה והגדרת הפער המחקרי	9
פרק 3 – מטרות, דרישות ואילוצים	10
3.1 מטרות הפרויקט	10
3.2 דרישות פונקציונליות	10
3.3 דרישות לא-פונקציונליות ואילוצים	11
3.4 קריטריוני הצלחה	11
פרק 4 – שיטת העבודה והגישה ההנדסית	12
4.1 גישת הפיתוח ההדרגתי (Incremental Development)	12
4.2 תכן משולב חמרה-תוכנה (HW/SW Co-Design)	12
4.3 מתודולוגיית אימות: מסימולציה לחמרה	12
4.4 כלים וסביבת עבודה	12
פרק 5 – תכן המערכת	14
5.1 ארכיטקטורת המערכת הכוללת	14
5.2 צנרת העיבוד החד-ערוצית (TOP)	14
5.3 לוגיקת רב-ערוציות: Output MUX ו-matcher, selector	15
5.4 פרוטוקול AXI-Stream ומבנה מילת הפלט	15
5.5 משוואות וחישובים	16

פרק 6 – מימוש	18
6.1 מימוש ה-RTL: מודולים עיקריים	18
6.2 מחולל החבילות לבדיקה עצמית (fsk_packet_generator)	18
6.3 תכן Block Design ב-Vivado ואינטגרציה עם Zynq PS	19
6.4 תוכנת ה-run_fpga.py: PS (DMA וויזואליזציה)	19
פרק 7 – ניסויים, בדיקות ואימות	20
7.1 בדיקות סימולציה	20
7.2 סגירת תזמון (Timing Closure)	20
7.3 בדיקת חמרה על PYNQ-Z2	20
7.4 תרחישי בדיקה	21
פרק 8 – תוצאות ודיון	22
8.1 ניצול משאבי החמרה (Resource Utilization)	22
8.2 תוצאות תזמון	22
8.3 תוצאות פונקציונליות	22
8.4 דיון ומגבלות	22
פרק 9 – מסקנות והמשך עבודה	24
9.1 סיכום הישגים	24
9.2 מגבלות	24
9.3 עבודה עתידית	24
מקורות ייחוס	25
מקורות ושימוש בכלי AI	25
נספחים	26
נספח א - רשימת מודולי RTL ותיאור פורטים	26
נספח ב - מבנה תוכנת ה-(run_fpga.py) PS	26

רשימת תרשימים

תרשים 1: ארכיטקטורת המערכת הכוללת: ערוצי עיבוד, מנגנוני השוואה/בחירה, ריבוב פלט וממשקי AXI אל ה-PS.....	14
תרשים 2: שרשרת העיבוד החד-ערוצית: מ-SampleInput ועד Output_Interface.....	15
תרשים 3: מבנה מילת הפלט (64 סיביות) של multi_channel_top.....	16

רשימת טבלאות

טבלה 1: דרישות פונקציונליות עיקריות של המערכת.....	10
טבלה 2: כלים וסביבת העבודה בפרויקט.....	12
טבלה 3: תגיות סוג החבילה (Output_Interface) ותכולת המטען.....	16
טבלה 4: תקציר תזמון התכן הסופי (Routed Design Timing Summary).....	20
טבלה 5: תרחישי בדיקה ותוצאותיהם.....	21
טבלה 6: ניצול משאבי FPGA לאחר (7z020clg400-1, design_1_wrapper) Place & Route.....	22
טבלה 7: רשימת מודולי RTL עיקריים בפרויקט.....	26
טבלה 8: פונקציות ומחלקות מרכזיות ב-run_fpga.py.....	26

פרק 1 – מבוא

1.1 רקע כללי ותחום הפרויקט

תקשורת אלחוטית היא אחד מתחומי הטכנולוגיה המרכזיים של העשורים האחרונים, והיא מהווה תשתית עבור מערכות סלולריות, רשתות חיישנים, מערכות ניווט, ניטור תעשייתי ויישומי האינטרנט של הדברים (IoT). בבסיס מערכות אלו עומד תהליך אפנון האות – המרת מידע דיגיטלי לאות אנלוגי הניתן לשידור בתווך. אחת משיטות האפנון הבסיסיות והעמידות ביותר היא אפנון תדר בינארי (Binary Frequency Shift Keying, FSK), שבו ערכי הסיביות מיוצגים על ידי שני תדרי נשא שונים. בשל פשטותה ועמידותה בפני הפרעות, שיטת FSK נמצאת בשימוש נרחב בחיישנים אלחוטיים, מערכות טלמטריה, RFID פעיל ותקשורת נתונים בקצב נמוך.

מימוש מקלט FSK בזמן אמת מחייב שרשרת עיבוד אותות הכוללת דגימה, גילוי נוכחות אות (Energy Detection), המרה למישור התדר (FFT), קבלת החלטה על ערך הסיבית, סנכרון מסגרות וחילוף שדות מידע מתוך המסגרת. ביצוע שרשרת זו בזמן אמת, עבור מספר ערוצי קליטה במקביל, מחייב משאבי חישוב מקבילים – יתרון מובהק של מערכי שער מתוכנתים (Field-Programmable Gate Array, FPGA) על פני מעבדים כלליים. פלטפורמות מסוג System-on-Chip (SoC) המשלבות FPGA עם מעבד מוטבע, כגון משפחת Xilinx Zynq, מאפשרות לחלק את העומס בין חמרה לוגית (Programmable Logic, PL) המבצעת עיבוד אותות בקצב גבוה, לבין מעבד תוכנה (Processing System, PS) המבצע בקרה, ניתוח וויזואליזציה.

1.2 חשיבות הנושא ומגמות עכשוויות

הביקוש למערכות קליטה רב-ערוציות גדל בהתמדה: רשתות חיישנים מבוזרות, מערכי אנטנות, ומערכות ניטור ספקטרום נדרשות לעבד מספר אפיקי קליטה בו-זמנית, להשוות מידע בין ערוצים ולבחור את המקור האמין ביותר. במקביל, פלטפורמות כגון PYNQ-Z2 (המבוססת על Zynq-7020) הפכו את הפיתוח עבור FPGA לנגיש יותר, באמצעות מסגרת עבודה מבוססת Python המאפשרת טעינת bitstream, ניהול ערוצי AXI DMA וכתובת קוד ניתוח וויזואליזציה בסביבת Jupyter, ללא צורך בכלי פיתוח ייעודיים בצד המעבד.

מגמה נוספת היא הדגש על "נראות" (observability) של מערכות עיבוד אותות בזמן הרצה: יכולת להציג בזמן אמת את מצב הערוצים, ההספק הנמדד בכל תדר, אירועי סנכרון וכתובות שחולצו, מסייעת רבות בפיתוח, בכוון ובאבחון תקלות. שילוב של עיבוד אותות בחמרה לוגית עם תצוגה גרפית אינטראקטיבית בצד המעבד הוא לפיכך מאפיין רצוי במערכות מחקר ופיתוח מודרניות.

1.3 מצב קיים וטכנולוגיות דומות

קיימים פתרונות רבים למקלטי FSK חד-ערוציים, המבוססים על גלאי אנרגיה, מתאמים (correlators) ומפענחי FFT. פלטפורמות רדיו מוגדר תוכנה (Software-Defined Radio, SDR) כגון USRP בשילוב GNU Radio מציעות גמישות רבה, אך מבוססות במידה רבה על עיבוד תוכנה במעבד כללי, מה שמגביל את קצב העיבוד ומגדיל את ההשהיה ואת צריכת ההספק בהשוואה למימוש לוגי ייעודי. מצד שני, מימושי FPGA ייעודיים לרוב מתמקדים בערוץ קליטה בודד, ומספקים ממשק פלט מצומצם (לדוגמה דגל בודד או רישום AXI-Lite), ללא שכבת השוואה בין ערוצים וללא תצוגה גרפית מובנית.

בספריית הפרויקטים עבור PYNQ קיימות הדגמות בסיסיות ל-DMA, ל-IP FFT ולעיבוד אותות פשוט, אך אלו משמשות בעיקר כדוגמאות לימודיות ואינן כוללות שרשרת עיבוד FSK מלאה, ואינן מספקות מנגנון רב-ערוצי המשווה בין כתובות שחולצו מערוצים שונים ובוחר ערוץ מועדף על בסיס מדד אמינות (לדוגמה מרווח הספק בין שני תדרי ה-FSK).

1.4 הפער הקיים והבעיה הנדונה

מתוך הסקירה לעיל עולה פער ברור: אין בנמצא מימוש משולב, נגיש וזול, של מקלט FSK רב-ערוצי על גבי FPGA, המבצע לכל ערוץ שרשרת עיבוד אותות מלאה (גילוי אנרגיה, FFT, החלטת סיבית, סנכרון וחילוף כתובות), משווה בין הכתובות שחולצו מהערוצים השונים, בוחר ערוץ מועדף על בסיס מדד הספק, ומעביר את כל המידע הזה בזמן אמת לתצוגה גרפית בצד המעבד – הכולל על גבי לוח PYNQ-Z2 בעלות נמוכה. הבעיה המרכזית שהפרויקט מתמודד עמה היא, אם כן: כיצד ניתן לתכנן ולממש מערכת כזו, באופן שמירבי משאבי

החמרה, עומד בדרישות התזמון של ה-FPGA, ומאפשר הדגמה מעשית של גילוי איתות, סנכרון, חילוץ כתובת והשוואה בין ערוצים – הכול בזמן אמת ובתצוגה נגישה.

1.5 מטרת הפרויקט ומבנה הספר

מטרת הפרויקט היא לתכנן, לממש ולאמת מערכת FPGA רב-ערוצית (שני ערוצים, הניתנת להרחבה) לזיהוי אותות FSK, הכוללת מחולל אותות לבדיקה עצמית, מנגנון השוואת כתובות בין ערוצים, מנגנון בחירת ערוץ מועדף, ושכבת תקשורת מבוססת AXI-Stream ו-AXI DMA אל מעבד ה-PS, שם מוצגים הנתונים בזמן אמת בסביבת Jupyter על גבי PYNQ-Z2.

יתר חלקי הספר בנויים כך: פרק 2 סוקר את הרקע התיאורטי והספרות הרלוונטית – אפנון FSK וגילוי אנרגיה, עיבוד אותות מבוסס FFT על FPGA, סנכרון מסגרות וחילוץ כתובת, קבלה רב-ערוצית ובחירת ערוץ, ופלטפורמות Zynq/PYNQ. פרק 3 מגדיר את מטרות הפרויקט, הדרישות והאילוצים. פרק 4 מתאר את שיטת העבודה ההנדסית. פרק 5 מציג את תכן המערכת – הארכיטקטורה, צורת העיבוד, מנגנוני ההשוואה והבחירה, ופרוטוקול הפלט, לרבות המשוואות המרכזיות. פרק 6 מתאר את המימוש בפועל – RTL, מחולל החבילות, תכן ה-Block Design ותוכנת ה-PS. פרק 7 מתאר את הבדיקות והאימות – סימולציה, סגירת תזמון ובדיקת חמרה. פרק 8 מציג את התוצאות ודן בהן, ופרק 9 מסכם את הפרויקט ומציע כיווני המשך.

פרק 2 – סקירת ספרות ורקע תיאורטי

2.1 אפנון FSK ועקרונות גילוי אנרגיה

אפנון תדר בינארי (BFSK) מייצג כל סיבית באמצעות אחד משני תדרי נשא, f_0 ו- f_1 . בצד המקלט, ניתן להעריך אילו מהתדרים מועבר על ידי השוואת ההספק הספקטרי באגף התדר המתאים לכל אחד מהתדרים: אם ההספק באגף המתאים ל- f_1 גבוה מההספק באגף המתאים ל- f_0 , מוסקת סיבית "1", ולהפך [1], [7]. שיטה זו אינה רגישה לאפקטים של אפנון פאזה ולכן עמידה ביישומים מעשיים.

לפני ביצוע ניתוח ספקטרי יקר חישובית, נהוג להשתמש בגלאי אנרגיה (Energy Detector) כשלב מקדים שמטרתו להבחין בין נוכחות אות לבין רעש בלבד. גלאי אנרגיה קלאסי מחשב את סכום ריבועי הדגימות בחלון זמן נתון ומשווה אותו לסף המבוסס על הספק הרעש הצפוי [6]. עבודתו של Urkowitz מהווה בסיס תיאורטי לקביעת ספי גילוי ולניתוח הסתברויות הגילוי וההתראת השווא של גלאי אנרגיה. בפרויקט הנוכחי מיושם גלאי אנרגיה דומה כשלב סינון מקדים, המונע הפעלת חישוב FFT בזמן שאין אות בעל אנרגיה משמעותית.

2.2 עיבוד אותות מבוסס FFT על FPGA

התמרת פורייה המהירה (FFT) היא הכלי המרכזי להמרת אות מהמישור הזמני למישור התדר, ומאפשרת חישוב הספק עבור כל אגף תדר (bin) בנפרד. עבור מערכות FSK, אין צורך בחישוב הספקטרום המלא – מספיק לחשב את ההספק בשני אגפי תדר המתאימים ל- f_0 ול- f_1 . עם זאת, מימוש חמרי יעיל של FFT הוא משימה מורכבת, ולכן נהוג להשתמש בליבות IP מוכנות, כגון ליבת FFT של Xilinx (LogiCORE IP (Fast Fourier Transform), המספקת ממשק AXI-Stream סטנדרטי, תצורת Pipelined Streaming I/O, וטיפול אוטומטי בהיפוך סדר הביטים (bit-reversal) ובאינדוקס הפלט [3].

שימוש בליבת IP מוכנה מאפשר למתכנן להתמקד בלוגיקת הבקרה (FFT controller), בבחירת אגפי התדר הרלוונטיים (Bin Selector) ובחישוב ההספק מתוך המקדמים המרוכבים שמתקבלים, $P[k] = \text{Re}[k]^2 + \text{Im}[k]^2$, ולא בפיתוח אלגוריתם ה-FFT עצמו. גישה זו מקצרת מאוד את זמן הפיתוח ומבטיחה ביצועים יציבות נומריים מוכחים, ומיושמת גם בפרויקט הנוכחי.

2.3 סנכרון מסגרות (Sync Detection) וחילוץ כתובת

מסגרות תקשורת דיגיטליות נפתחות לעיתים קרובות במילת סנכרון (Preamble/Sync Word) קבועה וידועה מראש, המאפשרת למקלט לזהות את נקודת ההתחלה של המסגרת ולהתאים את שעון הדגימה. שיטה נפוצה לגילוי מילת סנכרון היא רגיסטר הסטה (shift register) בגודל מילת הסנכרון, שבו כל סיבית חדשה שמתקבלת נדחפת לתוך הרגיסטר, והתוכן מושווה בכל שעון למילת הסנכרון הצפויה [1], [7]. אם מתקבלת התאמה מלאה (או התאמה מעל סף מסוים, המאפשרת סבילות לשגיאות סיביות בודדות), מוסק שגילוי הסנכרון אירע, ולאחר מכן הסיביות הבאות מתפרשות לפי מבנה המסגרת המוגדר – לדוגמה שדה כתובת באורך קבוע.

חילוץ שדה הכתובת מתבצע על ידי מנגנון מצב (Finite State Machine) הנכנס למצב "אוסף סיביות" לאחר גילוי הסנכרון, אוסף את מספר הסיביות המוגדר לשדה הכתובת, ומאתת תקפות (valid) בסיום האיסוף. ארכיטקטורה זו מיושמת באופן ישיר במודולים Sync_Detector ו-Adders_Extractor בפרויקט הנוכחי.

2.4 קבלה רב-ערוצית, בחירת ערוץ וגיוון (Diversity)

מערכות קליטה רב-ערוציות נפוצות בתקשורת אלחוטית כאמצעי לשיפור אמינות הקליטה (גיוון מרחבי/תדרי) או כאמצעי לקליטה מבוזרת ממספר מקורות במקביל. בשיטת "בחירת גיוון" (Selection Diversity) נבחר הערוץ בעל יחס אות-לרעש הגבוה ביותר לצורך קבלת ההחלטה הסופית [1], [9]. עיקרון דומה מיושם בפרויקט הנוכחי באמצעות מודול channel_selector, הבוחר את הערוץ בעל מרווח ההספק הגדול ביותר בין שני אגפי התדר ($P_0 - P_1$), כמדד לרמת הביטחון בהחלטת ה-FSK.

בנוסף לבחירת ערוץ מועדף, ביישומים שבהם מספר מקורות עשויים לשלוח את אותה כתובת (לדוגמה התקן המשדר באופן סימולטני למספר אנטנות, או רשת חיישנים שבה כתובת מטרה משותפת מצופה להתקבל במספר ערוצים), יש ערך להשוואה בין הכתובות שחולצו מערוצים שונים וגילוי התאמה (match). מודול

channel_matcher בפרויקט הנוכחי מבצע השוואה כזו, ומסמן match_detected "bit" כאשר שתי כתובות שחולצו מערוצים שונים זהות.

2.5 פלטפורמות Zynq/PYNQ ותכן משולב חמרה-תוכנה (HW/SW Co-Design)

משפחת ה-Zynq-7000 של Xilinx משלבת מעבד ARM Cortex-A9 כפול-ליבות (Processing System,) עם מערך לוגיקה מתוכנתת (Programmable Logic, PL) על אותו רכיב שבב, המקושרים ביניהם באמצעות מערך ממשקי [10], [2] AMBA AXI. ארכיטקטורה זו מאפשרת חלוקת עבודה טבעית: ה-PL מבצע עיבוד אותות מקבילי בקצב שעון גבוה, ואילו ה-PS מספק בקרה, ניטור, אחסון וממשק משתמש. העברת נתוני מסה בין PL ל-PS מתבצעת ביעילות באמצעות בקר AXI DMA, המאפשר העברת זרמי AXI-Stream לאזורי זיכרון בלי מעורבות מתמדת של המעבד [4].

פלטפורמת PYNQ (Python Productivity for Zynq) מספקת שכבת תוכנה מבוססת Python ו-Jupyter Notebook להפעלת bitstreams, ניהול overlay-ים, גישה לערוצי AXI DMA והפעלת רישומי בקרה מסוג AXI-GPIO, ובכך מקצרת משמעותית את זמן הפיתוח והבדיקה של מערכות PL/PS משולבות [5]. לוח PYNQ-Z2, המבוסס על רכיב XC7Z020, מהווה פלטפורמת חמרה זמינה ובעלת עלות נמוכה המתאימה לפרויקטי מחקר ולימוד, ושימשה כפלטפורמת המטרה של הפרויקט הנוכחי.

2.6 סיכום הסקירה והגדרת הפער המחקרי

הסקירה לעיל מציגה את אבני הבניין התיאורטיות הקיימות בנפרד – גילוי אנרגיה, עיבוד FFT חמרתי, גילוי סנכרון וחילוף שדות, וכן עקרונות גיוון ובחירת ערוץ במערכות רב-ערוציות, ופלטפורמות Zynq/PYNQ לתכן משולב. עם זאת, השילוב המלא של כל המרכיבים הללו לכדי מערכת FSK רב-ערוצית אחת, הכוללת לכל ערוץ שרשרת עיבוד מלאה, מנגנון השוואת כתובות בין ערוצים, מנגנון בחירת ערוץ מועדף, ותצוגה גרפית בזמן אמת בצד ה-PS – אינו מתועד באופן נגיש כפתרון אחוד עבור לוח PYNQ-Z2. פער זה מהווה את נקודת המוצא להגדרת מטרות הפרויקט ודרישותיו, המפורטות בפרק הבא.

פרק 3 – מטרות, דרישות ואילוצים

3.1 מטרות הפרויקט

מטרת-העל של הפרויקט היא תכן, מימוש ואימות של מערכת FPGA רב-ערוצית לזיהוי אותות FSK על גבי לוח PYNQ-Z2. ממטרת-על זו נגזרות המטרות הספציפיות הבאות:

1. מימוש שרשרת עיבוד אותות חד-ערוצית מלאה (גלאי אנרגיה, FFT, בחירת אגפי תדר, חישוב הספק, החלטת FSK, גילוי סנכרון וחילוץ כתובת).
2. הרחבת השרשרת החד-ערוצית למבנה רב-ערוצי פרמטרי (N_channel), עם ריבוב פלט משולב לזרם AXI-Stream יחיד.
3. מימוש מודול השוואת כתובות בין ערוצים (channel_matcher) המסמן "match_detected" כאשר שני ערוצים מחלצים כתובת זהה.
4. מימוש מודול בחירת ערוץ מועדף (channel_selector) על בסיס מרווח ההספק בין שני אגפי התדר.
5. מימוש מקור אותות פנימי לבדיקה עצמית (fsk_packet_generator), המאפשר אימות מלא של המערכת ללא חמרת RF חיצונית.
6. שילוב המערכת בתכן Block Design הכולל את ה-AXI DMA, PS, ורישומי בקרה (AXI-GPIO) לסף האנרגיה, אגפי התדר ומספר הערוצים הפעילים.
7. כתיבת תוכנת PS (Python/PYNQ) הקוראת את הזרם באמצעות DMA, מפענחת אותו ומציגה תצוגת ניטור גרפית בזמן אמת.
8. אימות התכן בסימולציה ובחמרה, תוך עמידה בדרישות תזמון (timing closure) של ה-FPGA.

3.2 דרישות פונקציונליות

טבלה 1: דרישות פונקציונליות עיקריות של המערכת

#	דרישה
1	המערכת תזהה נוכחות אות בעל אנרגיה מעל סף מוגדר (min_energy) לכל ערוץ.
2	המערכת תחשב הספק בשני אגפי תדר (bin_f0, bin_f1) באמצעות FFT בגודל 256.
3	המערכת תקבע ערך סיבית (bit) על סמך השוואת ההספק בין שני האגפים.
4	המערכת תזהה מילת סנכרון באורך 16 סיביות ותתחיל איסוף כתובת לאחריה.
5	המערכת תחלץ שדה כתובת באורך 7 סיביות מכל ערוץ.
6	המערכת תשווה כתובות שחולצו מערוצים שונים ותדווח על התאמה (match_detected).
7	המערכת תבחר ערוץ מועדף לפי מרווח ההספק הגדול ביותר (best_channel, best_valid).
8	המערכת תרכיב מילת פלט אחת (64 סיביות) הכוללת תגית סוג, מטען, ומידע רב-ערוצי, ותשדר אותה ב-AXI-Stream.
9	תוכנת ה-PS תקבל את הזרם באמצעות AXI DMA, תפענח את שדות המילה ותציג גרפים מתעדכנים בזמן אמת.
10	מספר הערוצים הפעילים יהיה ניתן לקביעה בזמן ריצה באמצעות רישום (n_active) AXI-GPIO.

3.3 דרישות לא-פונקציונליות ואילוצים

בנוסף לדרישות הפונקציונליות, הוגדרו האילוצים הבאים הנובעים מהפלטפורמה ומהיעדים ההנדסיים של הפרויקט:

- פלטפורמת מטרה: PYNQ-Z2 (רכיב Zynq XC7Z020-1CLG400C), שעון מערכת 100MHz.
- עמידה בדרישות תזמון: Worst Negative Slack (WNS) ו-Worst Hold Slack (WHS) חיוביים בדוח התזמון הסופי (Routed Design Timing Summary).
- ניצול משאבים סביר: שימוש בחלק קטן ממשאבי ה-FPGA (LUT, Register, BRAM, DSP) כדי לאפשר הרחבה עתידית למספר ערוצים גדול יותר.
- פרמטריזציה: מספר הערוצים ($N_{channel}$), רוחב הכתובת ($ADDR_LENGTH$), גודל ה-FFT ($windows=256$) ואורך מילת הסנכרון ($sync_length=16$) יוגדרו כ-generics הניתנים לשינוי.
- שימוש בליבות IP מאומתות של Xilinx (FFT) להפחתת סיכון פיתוח וזמן.
- אי-תלות בחמרת RF חיצונית לצורך בדיקה - מימוש מקור אות פנימי לבדיקה עצמית.
- שמירה על תאימות לאחור: שינויים ב-RTL לא יפגעו בצנרת הנתונים הקיימת ולא יגרמו לעצירה (stall) של ה-pipeline.

3.4 קריטריוני הצלחה

הפרויקט יחשב למוצלח אם יתקיימו כל אלה: (1) ה-bitstream נבנה ב-Vivado ועובר את שלב ה-Place & Route ללא שגיאות, עם WNS ו-WHS חיוביים; (2) המערכת נצרכת ללוח PYNQ-Z2 ומפיקה זרם AXI-Stream תקין הנקלט באמצעות (3) AXI DMA בהפעלה עם מחולל החבילות הפנימי, שני הערוצים מציגים גרפי הספק מתעדכנים עבור שני תדרי ה-FSK; (4) חבילת הכתובת הראשונה מניבה $match_detected=1$ בין הערוצים, וחבילות עוקבות מניבות כתובות בלתי-תלויות לכל ערוץ; (5) תצוגת הניטור הגרפית בצד ה-PS מתעדכנת באופן רציף וללא קיפאון, ומציגה את כל השדות הרלוונטיים (הספק, כתובות, אירועי סנכרון, מצב $match/best$).

פרק 4 – שיטת העבודה והגישה ההנדסית

4.1 גישת הפיתוח ההדרגתי (Incremental Development)

הפרויקט פותח בגישה הדרגתית: בשלב הראשון מומשה ואומתה שרשרת עיבוד אותות חד-ערוצית מלאה (ישות TOP), הכוללת את כל הבלוקים הפונקציונליים – מ-קלט הדגימה ועד ממשק הפלט. רק לאחר שהשרשרת החד-ערוצית אומתה בסימולציה, היא שוכפלה למבנה רב-ערוצי פרמטרי (multi_channel_top), שמעליו נוספו שכבות ההשוואה והבחירה (channel_matcher, channel_selector) וריבוב הפלט. גישה זו מאפשרת איתור תקלות בשכבה הנכונה – אם תקלה מופיעה רק במבנה הרב-ערוצי, ניתן לשלול מראש את הצנרת החד-ערוצית הבסיסית כמקור הבעיה.

4.2 תכן משולב חמרה-תוכנה (HW/SW Co-Design)

התכן כולו מתוכנן מראש כמערכת PS/PL משולבת: הלוגיקה המתוכנתת (PL) מבצעת את כל עיבוד האותות בזמן אמת ומפיקה זרם AXI-Stream יחיד עם מבנה מילה מתועד; ה-PS, באמצעות PYNQ, קורא את הזרם הזה ב-AXI DMA, ומבצע פענוח, אגרגציה (היסטוריית הספק, יומן כתובות, ספירת אירועים) והצגה גרפית. חלוקת תפקידים זו ממוקסמת כך שכל לוגיקה הדורשת קצב גבוה (FFT, חישובי הספק, מיון סיביות) נשארת ב-PL, ואילו כל לוגיקה הדורשת גמישות, פיתוח מהיר וויזואליזציה (Pythonic) עוברת ל-PS.

רישומי בקרה (min_energy, bin_f0, bin_f1, n_active) ממומשים כ-AXI-GPIO, ומאפשרים לתוכנת ה-PS לכוון את פרמטרי הגילוי (סף אנרגיה, אגפי תדר, מספר ערוצים פעילים) בזמן ריצה, ללא צריבה חוזרת של ה-bitstream.

4.3 מתודולוגיית אימות: מסימולציה לחמרה

כל מודול RTL פותח ואומת תחילה בסימולציה התנהגותית, תוך התבססות על הגדרות הממשקים והפרוטוקול שבין המודולים (לדוגמה תזמון tvalid/tready/tlast בין Output_Interface לבין multi_channel_top). לאחר אימות פונקציונלי, התכן עבר סינתזה ומימוש (Place & Route) ב-Vivado, ונבדק דוח התזמון (Timing Summary) – מטרת-העבודה היתה להבטיח Worst Negative Slack (WNS) ו-Worst Hold Slack (WHS) חיוביים, המעידים על תכן שעומד בדרישות התזמון בתדר השעון הנתון. רק לאחר עמידה בדרישות אלו נוצר bitstream וצורב ללוח PYNQ-Z2 בפועל לבדיקת חמרה.

מחזור הפיתוח כלל גם איטרציות תיקון: לדוגמה, שינוי במנגנון ייצור הכתובות במחולל החבילות (fsk_packet_generator) שגרם לעצירת ה-pipeline אותר באמצעות בדיקת חמרה (תקיעות בספירת המילים שהתקבלו ב-PS), ותוקן בגרסה מאוחרת יותר באופן ששמר על מבנה ה-pipeline הקיים ולא הוסיף לוגיקת מיתוג נוספת – עיקרון של "שינוי מינימלי נדרש" שהונחה את כל איטרציות התיקון בפרויקט.

4.4 כלים וסביבת עבודה

טבלה 2: כלים וסביבת העבודה בפרויקט

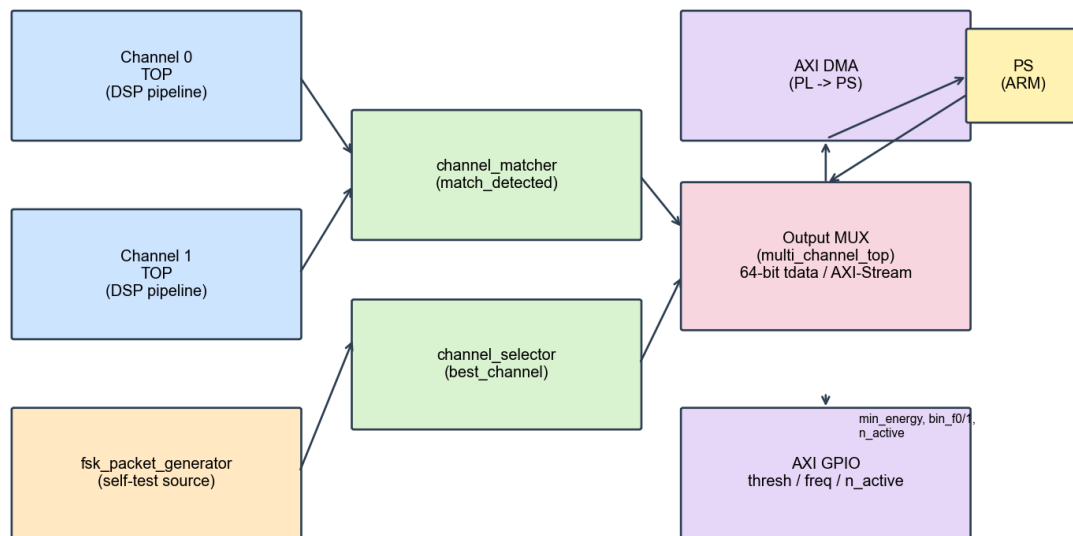
תפקיד בפרויקט	כלי / רכיב
סינתזה, Place & Route, יצירת bitstream ובניית ה-Block Design	Xilinx Vivado 2019.1
שפת תיאור החמרה למימוש כל מודולי ה-RTL	VHDL
ליבת IP לחישוב FFT בגודל 256 בממשק AXI-Stream	Xilinx LogiCORE FFT (xfft)
העברת זרם הפלט ל-PS ובקרת פרמטרים בזמן ריצה	AXI DMA, AXI-GPIO
פלטפורמת המטרה - PS (ARM) + PL (FPGA)	PYNQ-Z2 (Zynq XC7Z020)
תוכנת PS: DMA, פענוח חבילות, ויזואליזציה ב-Jupyter	Python, PYNQ, NumPy, Matplotlib
סביבת הרצה אינטראקטיבית על ה-PS	Jupyter Notebook

פרק 5 – תכן המערכת

5.1 ארכיטקטורת המערכת הכוללת

תרשים 1 מציג את ארכיטקטורת המערכת הכוללת. בצד ה-PL נמצאים שני מופעים (או יותר, בהתאם לפרמטר N_channel) של ישות TOP, כל אחד מבצע שרשרת עיבוד אותות מלאה לערוץ אחד. בנוסף, מודול fsk_packet_generator משמש כמקור אותות פנימי לבדיקה עצמית, ומחליף את כניסות sample_in/sample_valid החיצוניות. פלטי שני הערוצים (tdata, tvalid, tlast בני 36 סיביות כל אחד) מוזנים למודול channel_matcher (המפיק את הדגל match_detected) ולמודול channel_selector (המפיק את best_channel ו-best_valid). מודול ריבוב הפלט (Output MUX, בתוך multi_channel_top) מאחד את הנתונים למילת AXI-Stream אחת ברוחב 64 סיביות, המועברת ל-PS באמצעות AXI DMA. רישומי AXI-GPIO מאפשרים לתוכנת ה-PS לקבוע את סף האנרגיה, אגפי התדר ומספר הערוצים הפעילים.

תיצורע-בר PS/PL תרוטקטיכרא: תכרעמ פישרת



תרשים 1: ארכיטקטורת המערכת הכוללת: ערוצי עיבוד, מנגנוני השוואה/בחירה, ריבוב פלט וממשקי AXI אל ה-PS.

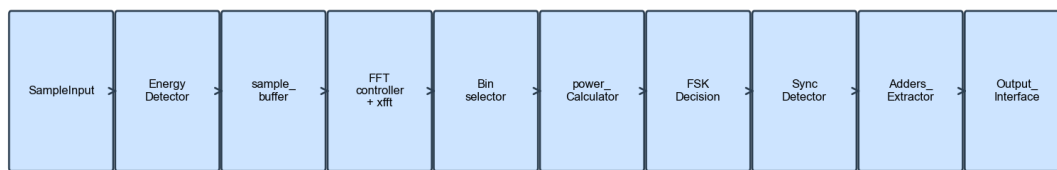
5.2 צנרת העיבוד החד-ערוצית (TOP)

תרשים 2 מציג את שרשרת העיבוד הפנימית של ישות TOP, המורכבת מהמודולים הבאים, בסדר מעבר הנתונים:

- SampleInput - רגיסטר קלט הקולט sample_in ו-sample_valid ומספק אותם לשלב הבא בעיבוד שרון אחד.
- Energy_Detector - מצבר אנרגיה על חלון של 256 דגימות; משווה את הסכום לסף min_energy ומפיק את הדגל energy_detected אחת לחלון.
- sample_buffer - חיץ זיכרון בגודל 256 דגימות, אוגר את הדגימות הנכנסות ומספק אותן ל-FFT controller לפי כתובת קריאה (read_addr); ממשיך לאגור חלון חדש רק אם אנרגיית החלון הקודם עברה את הסף (energy_detected=1).
- FFT_controller - מנגנון מצב המזין את 256 הדגימות לליבת ה-fft (Xilinx LogiCORE FFT) - xfft ליבת IP המבצעת FFT בגודל 256 על נתוני הקלט, ומפיקה זוגות (Re,Im) עבור כל אגף תדר, עם תגית tuser המציינת את אינדקס האגף.

- Bin_selector - אוגר את (Re,Im) של שני אגפי התדר הרלוונטיים (bin_f0, bin_f1) ומאתת bins_valid ומאתת bin_f0, bin_f1) ומאתת bins_valid כאשר שניהם נקלטו.
- power_Calculator - מחשב $P0 = Re^2 + Im^2$ ו- $P1 = Re^2 + Im^2$ ומאתת power_valid.
- FSK_Decision - קובע bit_out=1 אם $P0 > P1$, אחרת bit_out=0, ומאתת bit_valid.
- Sync_Detector - רגיסטר הסטה באורך 16 סיביות, משווה לקבוע SYNC ומאתת sync_detected כאשר מתקבלת התאמה מלאה.
- Adders_Extractor - לאחר sync_detected, אוסף 7 סיביות עוקבות לתוך adders_out ומאתת adders_valid בסיומן.
- Output_Interface - מנגנון מצב הממיר את האירועים (sync_detected, power0/power1, sync_detected,) לזרם AXI-Stream יחיד בעל תגיות סוג (36 סיביות), עם tlast בסוף כל חבילה.

(TOP.vhd) תיצורע-דח דוביע תרנצ מישרת



תרשים 2: שרשרת העיבוד החד-ערוצית: מ-SampleInput ועד Output_Interface.

5.3 לוגיקת רב-ערוציות: mux, selector ומatcher

מודול channel_matcher מקבל את כתובות כל הערוצים (adders_out, adders_valid) ומבצע השוואה רגישת-זמן: בכל פעם שמתקבלת כתובת חדשה מערוץ i, היא נשמרת ב-addr_latch(i) ומשווה לכתובות שנשמרו עבור כל הערוצים האחרים i=j!. אם נמצאת התאמה, נדלק הדגל match_detected, שהוא דגל "דביק" (sticky) הנשאר 1 עד reset. מנגנון זה מאפשר גילוי התאמה בין כתובות שהתקבלו בזמנים שונים (לא בהכרח בו-זמנית), כל עוד הכתובת הקודמת עדיין מאוחסנת ב-latch.

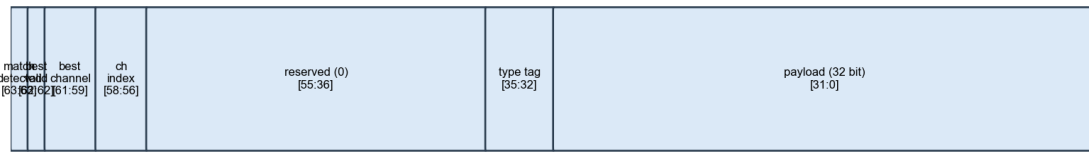
מודול channel_selector מחשב לכל ערוץ את מרווח ההספק $\text{margin}(i) = |P0(i) - P1(i)|$, ובכל פעם שמתקבל עדכון הספק תקף מערוץ כלשהו, בוחר את הערוץ עם המרווח הגדול ביותר מבין הערוצים שהתעדכנו. best_channel ו-best_valid הם אף הם דביקים, ומחזיקים את הערך האחרון שנקבע.

ה-Output MUX, המופעל מתוך multi_channel_top, מבצע round-robin בין הערוצים: בכל מחזור שעון, אם הערוץ הנוכחי (ch_index) מספק tvalid=1, מילת הפלט שלו (36 סיביות) מורחבת ל-56 סיביות (אפסים בביטים 36-55) ומשורשרת עם 8 ביטי המידע הרב-ערוצי - match_detected, best_valid, best_channel (3 סיביות) ו-ch_index (3 סיביות) - לכדי מילת פלט בת 64 סיביות. כאשר tlast_ch(ch_index)=1 וגם ch_index=1, ch_index מתקדם לערוץ הבא (מודולו n_active_int); אם הערוץ הנוכחי אינו פעיל (tvalid=0), ה-MUX מתקדם לערוץ הבא ללא עיכוב, כדי שערוץ פעיל לא יחסם מאחורי ערוץ שקט.

5.4 פרוטוקול AXI-Stream ומבנה מילת הפלט

מילת הפלט הסופית (64 tdata, 64 סיביות) המועברת ל-PS בנויה מהשדות הבאים (תרשים 3): ביט 63 - match_detected; ביט 62 - best_valid; ביטים 61-59 - best_channel; ביטים 58-56 - ch_index (מקור המילה); ביטים 55-36 - שמורים (אפס); ביטים 35-32 - תגית סוג (type tag); ביטים 31-0 - מטען (payload).

multi_channel_top לש (תויבס 64) טלפה תלימ הנבמ :םישרת



תרשים 3: מבנה מילת הפלט (64 סיביות) של multi_channel_top.

טבלה 3: תגיות סוג החבילה (Output_Interface) ותכולת המטען

ערוך תגית	משמעות	תוכן השדה payload
0001	power_0	הספק באגף התדר הראשון, (P0), 32 סיביות
0010	power_1	הספק באגף התדר השני, (P1), 32 סיביות
0011	sync	ביט 0 - דגל sync_detected
0100	adders (address)	ביטים 6-0 - שדה הכתובת שחולץ (7 סיביות)

חבילות power_0 ו-power_1 משודרות בזוג (חבילת 2 מילים) עם tlast בסוף power_1; חבילות sync ו-run_fpga.py הן חבילות בנות מילה אחת בלבד עם tlast=1. בצד ה-PS, פונקציית parse_word מחלצת את כל השדות הנ"ל באמצעות פעולות הסטה ומיסוך ביטים (bit shift / mask), ומעדכנת את מצב הערוץ המתאים בהתאם לתגית.

5.5 משוואות וחישובים

פרמטרי האפנון והעיבוד הספקטרי המרכזיים במערכת הם:

קצב הדגימה: $F_s = 1 \text{ MHz}$ (מקבל מחלוקת שעון $\text{CLK_DIV}=100$ משעון מערכת 100MHz במחולל החבילות לבדיקה עצמית).

גודל ה-FFT: $N_{\text{FFT}} = 256$ (דגימות $\text{windows}=256$), כלומר כל סיבית מיוצגת על ידי חלון של 256 דגימות ($\text{SAMPLES_PER_BIT}=256$).

רזולוציית תדר: $\Delta f = F_s / N_{\text{FFT}} = 1,000,000/256 \approx 3906.25 \text{ Hz}$

תדרי FSK: $f_0 (\text{bit}=0) = 120 \text{ kHz}$, $f_1 (\text{bit}=1) = 100 \text{ kHz}$, כפי שמוגדר במחולל החבילות ובהתאם למיפוי שב- $\text{FSK_Decision} (\text{power}_0 > \text{power}_1 \Rightarrow \text{bit}=1)$

אינדקס אגף ה-FFT לתדר נתון: $k = \text{round}(f / \Delta f)$. אגפי התדר bin_f0 ו- bin_f1 מוגדרים כרישומי AXI-GPIO בני 8 סיביות, ומאפשרים כיוון בזמן ריצה.

חישוב הספק: $P[k] = \text{Re}[k]^2 + \text{Im}[k]^2$, המבוצע על ידי power_Calculator לשני האגפים הנבחרים בלבד.

החלטת סיבית: $\text{bit} = 1$ אם $P_0 > P_1$, אחרת $\text{bit} = 0$ (FSK_Decision.vhd).

גילוי אנרגיה: $E = \sum x[n]^2$ על חלון של 256 דגימות; $\text{detected}=1$ אם $E > \text{min_energy}$ (רישום AXI-GPIO בן 32 סיביות).

מרווח הספק לבחירת ערוץ: $\text{margin}(i) = |P_0(i) - P_1(i)|$; $\text{best_channel} = \text{argmax}_i \text{margin}(i)$; הערוצים שהתעדכנו במחזור הנוכחי.

מחולל ה-DDS (במחולל החבילות לבדיקה עצמית) משתמש במצבר פאזה בן 16 סיביות וטבלת סינוס בגודל $\text{phase_inc}(\text{bit}=1) = \text{round}(65536 \cdot 100000/1e6) = 6554$, $\text{phase_inc}(\text{bit}=0) = 256$; $\text{round}(65536 \cdot 120000/1e6) = 7864$

מבנה החבילה הנשלחת על ידי מחולל הבדיקה העצמית הוא: PREAMBLE_BITS=4 חלונות של 120kHz
(להפעלת גלאי האנרגיה וסנכרון חלונות), ולאחריהם PKT_BITS = SYNC_LENGTH +
ADDR_LENGTH = 16 + 7 = 23 סיביות (מילת סנכרון ולאחריה שדה כתובת). בין חבילות מיוחדות
GAP_BITS=2 חלונות רעש בעלי אנרגיה נמוכה (NOISE_AMPLITUDE=200), הנמוכה דיה כדי שלא
תפעיל את גלאי האנרגיה.

פרק 6 – מימוש

6.1 מימוש ה-RTL: מודולים עיקריים

כל מודולי המערכת מומשו בשפת VHDL, עם generics מתאימים לפרמטריזציה (לדוגמה sample_size=16, windows=256, sync_length=16, adders_length=7, N_channel=2). מודול SampleInput הוא רגיסטר קלט פשוט בעיבוד מחזור שעון אחד. מודול Energy_Detector מצבר בר 40 סיביות (unsigned(39 downto 0)) את ריבועי הדגימות (32 סיביות לאחר הכפלה) על פני חלון של 256 דגימות, ומשווה לסף min_energy בסוף החלון. מודול sample_buffer מממש מערך (bufferType) בגודל 256 איברים, עם מנגנון "אם האנרגיה לא עברה את הסף, אפס את מחוון הכתיבה והתחל חלון חדש" - כך שחלונות שקטים אינם נשלחים ל-FFT.

מודול FFT_controller הוא מנגנון מצב-דו-מצבי (sending / not sending) המזין את ליבת ה-FFT דגימה אחר דגימה מתוך sample_buffer, ומאתת fft_last בדגימה ה-256 ו-fft_data_ready בסיומה - איתות זה משמש את sample_buffer לאיפוס מחוון הכתיבה והתחלת איסוף החלון הבא. מודול Bin_selector אוגר את הפלט המרוכב (Re,Im) של שני אגפי התדר הרלוונטיים מתוך זרם הפלט הרציף של ה-lookup FFT לפי fft_bin_index (שמגיע מ-tuser), ומאתת bins_valid כאשר שני האגפים נקלטו עבור אותו חלון.

מודול power_Calculator מבצע את חישוב P0, P1 סיביות, signed, אך בערכים מובטחים אי-שליליים מכיוון שהם סכום ריבועים) במחזור שעון אחד אחרי bins_valid. מודול FSK_Decision קובע את bit_out לפי ההשוואה P0>P1. מודול Sync_Detector מממש רגיסטר הסטה בן 16 סיביות עם הקבוע SYNC = "1100010111011000", ומאתת sync_detected בהתאמה מלאה. מודול Adders_Extractor הוא מנגנון מצב הנכנס למצב "אוסף" (collecting) בעקבות sync_detected, אוסף 7 סיביות לתוך adders_out בסדר LSB-first, ומאתת adders_valid בסיום.

מודול Output_Interface (36 סיביות) מממש את שכבת הפלט החד-ערוצית: מנגנון מצב (power_0/power_1/sync/adders) הקובע אילו נתונים יישלחו בכל מחזור, עם עדיפות > sync > adders > power. tlast מאותת כאשר state /= power_0 וגם tvalid_reg=1, כך ש-power_0 ו-power_1 נשלחים כחבילה דו-מילתית, ואילו sync ו-adders נשלחים כחבילות בנות מילה אחת.

בשכבה הרב-ערוצית, multi_channel_top משכפל N_channel מופעי TOP, מנתב את treddy בהתאם לערוץ הנבחר (treddy_ch), מבצע "הרחבה" (zero-extend) של פלט כל ערוץ מ-36 ל-56 סיביות, מחלץ כתובות (addr_arr, addr_valid_arr) ועוצמות (power0_ch, power1_ch, power_valid_ch) באופן קומבינטורי/רשום, ומפעיל את channel_matcher, channel_selector ואת ה-Output MUX כמתואר בפרק 5.

6.2 מחולל החבילות לבדיקה עצמית (fsk_packet_generator)

כדי לאמת את המערכת ללא חמרת RF חיצונית, מומש מודול fsk_packet_generator המחליף את כניסות sample_in/sample_valid המודול כולל טבלת סינוס (SINE_LUT) בגודל 256 איברים באמפליטודה 8000, ומחולל DDS (Direct Digital Synthesis) עם מצבר פאזה בן 16 סיביות לכל ערוץ. מנגנון המצב הראשי כולל שלושה מצבים: ST_GAP (רעש LFSR בעוצמה נמוכה בין חבילות), ST_PREAMBLE (טון 120kHz קבוע להפעלת גלאי האנרגיה ויישור חלונות) ו-ST_PACKET (שידור מילת הסנכרון ולאחריה שדה הכתובת, כל סיבית מתורגמת לתדר FSK מתאים באמצעות שינוי קצב הפאזה).

ייצור הכתובות מתבסס על רגיסטרי הסטה עם משוב לינארי (LFSR, Galois form), פולינום $x^8+x^6+x^5+x^4+1$ בני 8 סיביות, אחד לכל ערוץ. כל ה-LFSR-ים מאופסים לזרע התחלתי משותף (to_unsigned(1,8)). ערוץ 0 מתקדם עם פולינום משוב $x^8+x^6+x^5+x^4+1$, ואילו ערוצים 1..N-1 מתקדמים עם פולינום משוב שונה $(x^8+x^7+x^6+x^5+1)$. כתוצאה מכך, בחבילה הראשונה (לפני ההתקדמות הראשונה של ה-LFSR), כל הערוצים משדרים כתובת זהה - מה שמפעיל את match_detected - ואילו מהחבילה השנייה ואילך הכתובות מתפצלות לרצפים בלתי-תלויים (פסבדו-אקראיים) לכל ערוץ, בשל הפולינומים השונים. גישה זו מספקת תרחיש בדיקה דטרמיניסטי הן להתאמה והן להתפצלות, מבלי להוסיף

מולטיפלקסור או מצב מיוחד שעלול לגרום לעצירת ה-pipeline - שיקול שהתגלה כקריטי באיטרציה קודמת של המימוש, בה מנגנון "חבילה ראשונה מיוחדת" נפרד גרם לקיפאון הזרם אחרי 34 מילים.

מקור רעש נוסף (noise_lfsr), עצמאי לכל ערוץ ומאותחל לזרעים שונים $(i*37+5)$, מספק רעש בעוצמה $\text{NOISE_AMPLITUDE}=200$ בין חבילות, הנמוכה בהרבה מסף גלאי האנרגיה, כדי לדמות תקופות שקטות מציאותיות בין שידורים.

6.3 תכן Block Design ב-Vivado ואינטגרציה עם Zynq PS

התכן ב-Vivado כולל (design_1) Block Design המשלב את ה-Zynq Processing System (PS7), בקר AXI DMA (axi_dma_0) להעברת זרם ה-AXI-Stream מ-PL ל-PS, ושלושה רישומי AXI-GPIO (axi_gpio_ctrl, axi_gpio_thresh, axi_gpio_freq) המעבירים מה-PS ל-PL את הפרמטרים n_active, min_energy ו-b1/f0/bin. ישות multi_channel_top (וברירת המחדל - גם fsk_packet_generator) מחוברות כ-IP מותאם-אישית (custom IP wrapper) בתוך ה-Block Design. הבנייה מתבצעת באמצעות סקריפט build_multi_channel_design.tcl, המבנה אידמפוטנטי - הרצות חוזרות מזהות שהרכיבים קיימים ומדלגות על יצירתם מחדש.

בניית ה-bitstream מתבצעת באמצעות סקריפט rebuild_bitstream.tcl, המריץ ברצף `reset_run -> launch_runs synth_1 -> wait_on_run -> launch_runs impl_1 -to_step write_bitstream` ובודק שדוח ההתקדמות מגיע ל-100% וכי ה-log מציין "Bitstream build complete". פלטי הבנייה הרלוונטיים לפריסה הם design_1_wrapper.bit (ה-bitstream) ו-design_1.hwh (תיאור החמרה, hardware handoff), הנדרשים שניהם על ידי PYNQ לטעינת ה-overlay.

6.4 תוכנת ה-run_fpga.py (PS: DMA וויזואליזציה)

בצד ה-PS, סקריפט run_fpga.py (בתיקייה sw/multi_channel) טוען את ה-overlay (design_1_wrapper.bit), פותח ערוץ קליטה של AXI DMA (axi_dma_0) ומבצע בלולאה אינסופית: הפעלת transfer לתוך חיץ זיכרון ($\text{BUFFER_WORDS}=4$ מילים מסוג uint64), המתנה לסיום ה-transfer עם timeout ($\text{DMA_TIMEOUT}=5\text{s}$), ממומש בעזרת thread נפרד כדי לא לחסום את הלולאה), ופענוח כל מילה שאינה אפס באמצעות פונקציית parse_word - המחלצת את match_detected, best_valid, type tag, ch_index, best_channel ו-payload בעזרת הסטות ומיסוך ביטים.

מצב כל ערוץ נשמר במופע ChannelState, הכולל היסטוריית הספק מתגלגלת (deque) באורך ($\text{HISTORY_LEN}=60$) לשני התדרים, מספר אירועי סנכרון, ויומן כתובות ($\text{ADDR_LOG_LEN}=30$). פונקציות build_figure ו-update_plots בונות figure של Matplotlib עם שלוש שורות גרפים לכל ערוץ: היסטוריית הספק, עמודות ההספק הנוכחי (P_0 מול P_1) עם תווית מרווח, ויומן הכתובות האחרונות. שורת מצב עליונה (status banner) מציגה את מספר המילים שהתקבלו, הערוץ המועדף הנוכחי ומספר ההתאמות שהתגלו.

הצגת התרשים בזמן אמת מתבססת על רינדור כל פריים לקובץ PNG בזיכרון (io.BytesIO) והצגתו באמצעות IPython.display.Image בשילוב (display.clear_output(wait=True)) המחליפה את תוכן תא Jupyter בכל עדכון. גישה זו נבחרה לאחר שגישה מבוססת (matplotlib widget) ipympl גרמה לכך שהתצוגה התעדכנה רק לאחר רענון הדפדפן ולאחר מכן הפסיקה להתעדכן. קצב העדכון מוגבל ($\text{REDRAW_INTERVAL}=0.1\text{s}$, כ-10FPS) בהתבסס על זמן אמת ולא על מספר ה-transfers, כך שהתצוגה נשארת קריאה גם כאשר ה-FPGA משדר חבילות הספק ברצף צפוף ($\sim 256\mu\text{s}$).

פרק 7 – ניסויים, בדיקות ואימות

7.1 בדיקות סימולציה

השלב הראשון באימות כל מודול היה סימולציה התנהגותית, שבה הוזנו רצפי דגימות מבוקרים (טונים בתדרי f0/f1, רעש, ורצף סנכרון+כתובת ידוע) ונבדקו הפלטים הצפויים: הפעלת energy_detected מעל הסף, ערכי P0/P1 וההחלטה bit_out המתאימה להם, גילוי sync_detected בנקודת הזמן הצפויה, וחילוץ adders_out התואם לכתובת שהוזרקה. ברמת הצנרת המלאה (TOP), נבדק שהפלט (tdata/tvalid/tlast) עוקב אחר פרוטוקול ה-AXI-Stream המוגדר - tlast בסוף כל חבילה, ו-tvalid יורד כאשר אין מידע לשליחה.

ברמה הרב-ערוצית, נבדק כי channel_matcher מסמן match_detected כאשר שני ערוצים מקבלים את אותה כתובת (גם אם לא בו-זמנית), וכי channel_selector בוחר את הערוץ עם מרווח ההספק הגדול יותר. כן נבדק שה-Output MUX מבצע round-robin תקין בין הערוצים וכולל את כל שמונה ביטי המידע הרב-ערוצי (match_detected, best_valid, best_channel, ch_index) במילת הפלט.

7.2 סגירת תזמון (Timing Closure)

לאחר אימות פונקציוני, התכן עבר סינתזה ומימוש (Place & Route) ב-Vivado 2019.1 עבור רכיב 7z020clg400-1, בתדר שעון 100MHz. דוח התזמון הסופי (Design Timing Summary, Routed) של גרסת התכן הנוכחית מציג:

טבלה 4: תקציר תזמון התכן הסופי (Routed Design Timing Summary)

מדד	ערך	משמעות
WNS (Worst Negative Slack)	ns 0.485+	מרווח חיובי בכל נתיבי ה-Setup - אין הפרת תזמון
WHS (Worst Hold Slack)	ns 0.020+	מרווח חיובי בכל נתיבי ה-Hold - אין הפרת Hold
TNS / THS Failing Endpoints	0 / 0	אין נקודות קצה הפרות תזמון
WPWS (Worst Pulse Width Slack)	ns 3.750+	דרישות רוחב פולס שעון מתקיימות

ערכי slack חיוביים בכל הקטגוריות מעידים כי התכן עומד בדרישות התזמון בתדר השעון הנתון, ומאפשרים יצירת bitstream תקין. תרשים הניצול (table 7.3 בהמשך) מצביע על כך שקיים מרווח משאבים נרחב, התומך בהרחבה עתידית למספר ערוצים גדול יותר מבלי לפגוע בתזמון.

7.3 בדיקת חמרה על PYNQ-Z2

ה-bitstream (design_1_wrapper.bit) וקובץ תיאור החמרה (design_1_wrapper.hwh) הועברו ללוח PYNQ-Z2 (כתובת 192.168.2.99) באמצעות Jupyter Contents API (סקריפט upload_to_pynq.py), ונטענו כ-overlay בתוך run_fpga.py. בהפעלה ראשונה, המערכת הריצה את מחולל החבילות הפנימי לשני ערוצים, ותוכנת ה-PS קראה את הזרם באמצעות AXI DMA והציגה תצוגת ניטור גרפית. נמדדה זרימת מידע רציפה (ללא timeouts של ה-DMA) וערכי הספק תואמים לתדרי ה-FSK שהוזרקו.

במהלך הפיתוח אותרה תקלה שבה גרסת ביניים של מחולל החבילות (עם מנגנון "חבילה ראשונה מיוחדת" המבוסס על אות first_packet ומולטיפלקסור נוסף על ch_addr) גרמה לעצירת זרם הנתונים אחרי 34 מילים. התקלה אותרה על ידי מעקב אחר מספר המילים שהתקבלו ב-PS (total_words), ותוקנה בגרסה הסופית על ידי מעבר לזריעת LFSR משותפת עם פולינומי משוב שונים לכל ערוץ (ראו 6.2) - פתרון שאינו מוסיף לוגיקת מיתוג נוספת ולכן אינו פוגע בתזמון ה-pipeline.

7.4 תרחישי בדיקה

טבלה 5: תרחישי בדיקה ותוצאותיהם

#	תרחיש	תוצאה צפויה	תוצאה שנמדדה
1	הזרמת רעש בלבד (ST_GAP)	,energy_detected=0 אין חבילות sync/adders	תואם - רק חבילות power בעוצמה נמוכה
2	חבילה ראשונה (כתובות זהות)	match_detected=1 בשני הערוצים	תואם - match_detected נדלק לאחר חבילה 1
3	חבילות +2 (כתובות מתפצלות)	כתובות שונות לכל ערוץ, ללא match נוסף אלא אם מתרחשת התאמה אקראית	תואם - רצפים בלתי- תלויים
4	גילוי סנכרון	sync_detected=1 בנקודת הזמן התואמת למילת הסנכרון	תואם - מתועד ביומן tag=3 [PKT]
5	בחירת ערוץ מועדף	best_channel עוקב אחרי הערוץ עם מרווח הספק גדול יותר	תואם - best_valid=1 ועדכון רציף
6	הרצה רציפה (קצב מילים גבוה)	אין timeout ב-DMA, התצוגה מתעדכנת ברצף ללא תקיעה	תואם לאחר מעבר לרינדור PNG+IPython.display

פרק 8 – תוצאות ודיון

8.1 ניצול משאבי החמרה (Resource Utilization)

טבלה 6: ניצול משאבי FPGA לאחר (design_1_wrapper, 7z020clg400-1) Place & Route

משאב	בשימוש	זמין	אחוז ניצול
Slice LUTs	7,539	53,200	14.17%
Slice Registers	10,794	106,400	10.14%
Block RAM Tile	6	140	4.29%
DSP48E1	28	220	12.73%

ניצול המשאבים נמוך משמעותית מהקיבולת הכוללת של רכיב XC7Z020 בכל הקטגוריות (מתחת ל-15% עבור לוגיקה ורגיסטרים, ומתחת ל-13% עבור DSP ו-BRAM). מרווח זה מאפשר הרחבה עתידית - לדוגמה הוספת ערוצים נוספים ($N_channel > 2$), שכן כל ערוץ נוסף מוסיף מופע נוסף של TOP (כולל xfft) אך אינו צפוי לחצות את גבולות המשאבים הקיימים בקנה מידה סביר (3-4 ערוצים).

8.2 תוצאות תזמון

כמתואר בפרק 7.2, התכן הסופי עומד בדרישות התזמון עם $WNS = +0.485ns$ ו- $WHS = +0.020ns$, ללא נקודות קצה כושלות. נתון זה מאשר כי שעון המערכת (100MHz, תקופה 10ns) נתמך באופן מלא על ידי כל נתיבי הלוגיקה, כולל נתיבי ה-LFSR ושרשרת הרישומים הרב-ערוצית שנוספה בשכבת ה-matcher/selector/mux. מרווח ה-WHS הקטן (0.020ns) מצריך תשומת לב בעת שינויים עתידיים - כל הוספת לוגיקה משולבת (combinational) משמעותית עשויה לדרוש בחינה מחדש של נתיבי ה-Hold.

8.3 תוצאות פונקציונליות

בהרצת המערכת על PYNQ-Z2 עם מחולל החבילות הפנימי, תוכנת ה-PS קלטה זרם AXI-Stream רציף של חבילות power_0/power_1 (מתועדות באמצעות histogram של תגיות, tag_counts), כאשר כל 256 מיקרו-שניות (חלון FFT בודד בקצב דגימה 1MHz) מתקבל זוג חדש של ערכי הספק לכל ערוץ. גרפי ההיסטוריה המתגלגלת הציגו דומיננטיות מתחלפת של P0 ו-P1 בהתאם לרצף הסיביות שהוזרק (סנכרון + כתובת), בהתאמה לתדרי 120kHz/100kHz שהוגדרו במחולל.

חבילת הכתובת הראשונה הניבה ערך $match_detected = 1$ עבור שני הערוצים, כצפוי מהזריעה המשותפת של ה-LFSR. מהחבילה השנייה ואילך, יומני הכתובות של שני הערוצים (addr_log) הציגו ערכים מתפצלים ובלתי-תלויים, בהתאם לפולינומי המשוב השונים. מודול channel_selector עדכן את best_channel ו-best_valid בהתאם למרווח ההספק הגדול ביותר שנמדד, ושורת המצב העליונה בתצוגה הגרפית הציגה ערכים אלו לצד מספר ההתאמות הכולל וספירת המילים שהתקבלו (total_words).

בעיית התצוגה שאותרה במהלך הפיתוח - חוסר עדכון רציף של הגרפים בדפדפן (תופעה האופיינית לעיתים ל-backend מסוג ipynb ב-Jupyter) - נפתרה באמצעות מעבר לרינדור כל פריים כתמונת PNG והצגתה מחדש באמצעות IPython.display, מה שהבטיח עדכון רציף של התצוגה ללא תלות בסנכרון widget בין הדפדפן לליבת ה-Jupyter.

8.4 דיון ומגבלות

התוצאות מדגימות כי ניתן לממש מקלט FSK רב-ערוצי מלא, כולל גילוי אנרגיה, ניתוח ספקטרום, החלטת סיביות, סנכרון, חילוף כתובת, השוואה בין ערוצים ובחירת ערוץ מועדף, על גבי לוח PYNQ-Z2 בעלות נמוכה, תוך ניצול נמוך של משאבי החמרה ועמידה בדרישות התזמון. עם זאת, קיימות מספר מגבלות: ראשית, מחולל החבילות הפנימי (fsk_packet_generator) מספק תרחיש בדיקה דטרמיניסטי בלבד, ואינו מדמה תנאי תקשורת מציאותיים מלאים (כגון shift בתדר, רעש פאזה, או היסט בתזמון בין ערוצים).

שנית, מנגנון ההשוואה בין ערוצים (channel_matcher) מבוסס על latch דביק לכל ערוץ, ולכן מתאים להשוואת "הכתובת האחרונה שנקלטה" - אם נדרשת השוואה מבוססת-זמן (לדוגמה התאמה רק אם שתי הכתובות התקבלו בתוך חלון זמן מוגדר), יידרש מנגנון נוסף עם חותמות זמן (timestamps). שלישית, מודול channel_selector בוחר ערוץ מועדף לפי מרווח הספק בלבד, ואינו מתחשב במהימנות ארוכת-טווח (לדוגמה ממוצע נע של מרווחים) - מה שעשוי להוביל לרגישות לרעש בחלון בודד.

לבסוף, נמדדה רגישות מתודולוגית חשובה: שינויים קטנים בלוגיקת מחולל החבילות (כגון הוספת מולטיפלקסור על ch_addr) גרמו לעצירת ה-pipeline אחרי 34 מילים, אף שהשינוי לא נראה כקריטי מבחינה פונקציונלית. דבר זה מדגיש את הצורך לבחון כל שינוי RTL גם מבחינת השפעתו על תזמון ה-handshake בין מודולים (tvalid/tready/tlast), ולא רק מבחינת הלוגיקה הפונקציונלית הישירה.

פרק 9 – מסקנות והמשך עבודה

9.1 סיכום הישגים

בפרויקט זה תוכננה ומומשה מערכת FPGA רב-ערוצית לזיהוי אותות FSK על גבי לוח PYNQ-Z2, הכוללת שרשרת עיבוד אותות מלאה לכל ערוץ (גלאי אנרגיה, FFT, בחירת אגפי תדר, חישוב הספק, החלטת סיבית, גילוי סנכרון וחילוץ כתובת), שכבת השוואה בין ערוצים (channel_matcher), שכבת בחירת ערוץ מועדף (channel_selector), ושכבת ריבוב פלט המאחדת את כל המידע למילת AXI-Stream אחת בת 64 סיביות. נוסף לכך, פותח מודול מחולל חבילות פנימי המאפשר בדיקה עצמית מלאה של המערכת, כולל תרחיש דטרמיניסטי של התאמת כתובות בחבילה הראשונה והתפצלות לרצפים בלתי-תלויים בחבילות הבאות. התכן עומד בדרישות התזמון (WNS, WHS חיוביים) ונבדק בהצלחה על חמרה אמיתית, כולל תצוגת ניטור גרפית בזמן אמת בסביבת Jupyter.

9.2 מגבלות

כפי שנדון בפרק 8.4, המגבלות המרכזיות של המימוש הנוכחי הן: (1) מקור האות לבדיקה הוא מחולל פנימי דטרמיניסטי ולא אות RF אמיתי; (2) מנגנון ההשוואה בין ערוצים אינו מבוסס חותמות זמן; (3) בחירת הערוץ המועדף מתבססת על מדידה בודדת ולא על מיצוע ארוך-טווח; (4) המערכת נבדקה עבור שני ערוצים בלבד, ולא נמדדו בפועל ביצועי מערכת עבור $N_channel > 2$.

9.3 עבודה עתידית

- הרחבת מספר הערוצים ($N_channel$) ל-4 ומעלה, ובחינת ההשפעה על ניצול משאבים ועל תזמון.
- הוספת ממשק קלט אנלוגי אמיתי (ADC חיצוני או front-end RF) כתחליף למחולל החבילות הפנימי, לאימות בתנאי שטח.
- הוספת חותמות זמן (timestamps) למודול channel_matcher, לאפשר השוואת כתובות בתוך חלון זמן מוגדר בלבד.
- שיפור channel_selector באמצעות מיצוע נע (moving average) של מרווח ההספק, לשיפור עמידות בפני רעש רגעי.
- מדידת מדדי ביצועים סטטיסטיים - הסתברות גילוי (P_d), הסתברות התראת שווא (P_{fa}) ושיעור שגיאות סיביות (BER) - תחת תנאי SNR שונים.
- הוספת תמיכה בפרוטוקול מסגרת מורכב יותר (לדוגמה שדה אורך מטען משתנה, CRC לבדיקת שגיאות).
- מדידת צריכת הספק בפועל על PYNQ-Z2 בהרצה מתמשכת, והשוואה לפתרון מבוסס SDR/תוכנה.

מקורות ייחוס

- J. G. Proakis and M. Salehi, "Digital Communications," 5th ed., New York: McGraw-Hill, [1] 2008
- Xilinx Inc., "Zynq-7000 SoC Technical Reference Manual," UG585 [2]
- Xilinx Inc., "LogiCORE IP Fast Fourier Transform v9.1 Product Guide," PG109 [3]
- Xilinx Inc., "AXI DMA v7.1 LogiCORE IP Product Guide," PG021 [4]
- PYNQ Project Documentation, Xilinx/AMD, available: <https://www.pynq.io> [5]
- H. Urkowitz, "Energy detection of unknown deterministic signals," Proceedings of the [6] IEEE, vol. 55, no. 4, pp. 523-531, 1967
- B. Sklar, "Digital Communications: Fundamentals and Applications," 2nd ed., Prentice Hall, [7] 2001
- Xilinx Inc., "AXI Reference Guide," UG761 [8]
- J. H. Reed, "Software Radio: A Modern Approach to Radio Engineering," Prentice Hall, [9] 2002
- ARM Ltd., "AMBA AXI and ACE Protocol Specification," ARM IHI 0022 [10]
- Xilinx Inc., "Vivado Design Suite User Guide: Implementation," UG904 [11]

מקורות ושימוש בכלי AI

במהלך הפרויקט נעשה שימוש בכלי בינה מלאכותית (Claude, Anthropic) לסיוע בכתיבת קוד ה-RTL, אבחון תקלות תזמון ו-pipeline, כתיבת תוכנת ה-PS ב-Python, וכתיבת ספר פרויקט זה. כל קוד שנוצר בעזרת הכלי נבחן, נבדק בסימולציה ובחמרה בפועל, ועודכן על פי תוצאות הבדיקות בידי הסטודנט. האחריות התוכנית והמדעית המלאה על התכן, המימוש והתוצאות המוצגות בספר זה היא של הסטודנט.

נספחים

נספח א - רשימת מודולי RTL ותיאור פורטים

טבלה 7: רשימת מודולי RTL עיקריים בפרויקט

מודול	תיאור קצר
SampleInput	רגיסטר קלט (עיכוב מחזור שעון אחד) לדגימה ולדגל תקפות
Energy_Detector	גלאי אנרגיה - מצבר על חלון 256 דגימות והשוואה ל-min_energy
sample_buffer	חיץ 256 דגימות עבור חלון FFT, עם בקרת מילוי מחדש
FFT_controller	מנגנון מצב להזנת ליבת ה-FFT ואיתות סיום חלון
xfft_0	ליבת IP של Xilinx לחישוב FFT בגודל 256 (AXI-Stream)
Bin_selector	בחירת שני אגפי תדר (Re,Im) מתוך פלט ה-FFT
power_Calculator	חישוב הספק, $P0=Re0^2+Im0^2$, $P1=Re1^2+Im1^2$
FSK_Decision	קביעת ערך סיבית לפי השוואת P0 ל-P1
Sync_Detector	גילוי מילת סנכרון (רגיסטר הסטה 16 סיביות)
Adders_Extractor	חילוץ שדה כתובת (7 סיביות) אחרי גילוי סנכרון
Output_Interface	מיתוג חבילות power/sync/adders לזרם AXI-Stream (36 סיביות)
TOP	שרשרת עיבוד חד-ערוצית מלאה - מאחדת את כל המודולים לעיל
fsk_packet_generator	מקור אותות פנימי לבדיקה עצמית (DDS + LFSR)
channel_matcher	השוואת כתובות בין ערוצים והפקת match_detected
channel_selector	בחירת ערוץ מועדף לפי מרווח הספק (best_channel/best_valid)
multi_channel_top	שכבה רב-ערוצית - שכפול TOP, מיתוג פלט והרכבת מילת 64 סיביות

נספח ב - מבנה תוכנת ה-run_fpga.py PS

טבלה 8: פונקציות ומחלקות מרכזיות ב-run_fpga.py

פונקציה / מחלקה	תפקיד
parse_word(word)	פענוח מילת פלט 64 סיביות לשדותיה (match/best/ch_index/tag/data)
ChannelState	מצב לכל ערוץ: היסטוריית הספק, ספירת סנכרונים, יומן כתובות
build_figure(n_ch)	בניית figure של Matplotlib עם תצי גרפים לכל ערוץ ושורת מצב
(...)update_plots	עדכון כל הגרפים על בסיס מצב הערוצים העדכני

המתנה ל-DMA עם timeout (thread נפרד)	dma_wait_timeout(...)
הלולאה הראשית - טעינת DMA, overlay, פענוח, רינדור	run(n_channels, bitstream)